IAI - TOGO / 2025 - 2026

INITIATION AUX BASES DE DONNEES



Dispensé par

M. N'SOUGAN FOLLY WOEDE

SOMMAIRE

Chapitre 1 : Introduction aux bases de données

I- Notions essentielles

II- Les différents modèles de bases de données

III- Niveaux d'abstraction des données

Chapitre 2 : La modélisation conceptuelle : Le modèle entité/association (E/A)

I- Éléments constitutifs du modèle Entité/Association

II- Types d'associations

Chapitre 3 : Modèle relationnel

I- Présentation

II- Éléments du modèle relationnel

Chapitre 4 : Manipulation des données : Algèbre relationnelle

I- Introduction

II- Les opérateurs

Chapitre 5 : Introduction au langage SQL

I- Présentation du Langage SQL

II- Structure des instructions SQL

III- Conventions de noms des objets de base de données

Chapitre 1 : Introduction aux bases de données

Les bases de données (BD) sont nées à la fin des années 1960 pour combler les lacunes des systèmes de fichiers et faciliter la gestion qualitative et quantitative des données informatiques. Les SGBD sont des applications informatiques permettant de créer et de gérer des BD (comme MySQL ou Oracle ou PostgreSQL par exemple)

Les BD relationnelles, issues de la recherche de Codd, sont celles qui ont connu le plus grand essor depuis les années, et qui reste encore aujourd'hui les plus utilisées. On utilise des SGBDR pour les implémenter. Le langage SQL est le langage commun à tous les SGBDR, ce qui permet de concevoir des BD relativement indépendantes des systèmes utilisés.

Les usages de BD se sont aujourd'hui généralisés pour entrer dans tous les secteurs de l'entreprise, depuis les petites bases utilisées par quelques personnes dans un service pour des besoins de gestion de données locales, jusqu'aux bases qui gèrent de façon centralisée des données partagées par tous les acteurs de l'entreprise.

L'accroissement de l'utilisation du numérique comme outil de manipulation de toutes données (bureautique, informatique applicative, etc.) et comme outil d'extension des moyens de communication (réseaux) ainsi que les évolutions technologiques (puissance des PC, Internet, etc.) ont rendu indispensable, mais aussi complexifié la problématique des BD.

Les conséquences de cette généralisation et de cette diversification des usages se retrouvent dans l'émergence de solutions conceptuelles et technologiques nouvelles, les bases de données du mouvement NoSQL particulièrement utilisées par les grands acteurs du web.

I- Notions essentielles

1- Qu'est qu'une donnée ?

Une donnée est un élément brut, qui n'a pas encore été interprétée, ou mis en contexte. Une donnée est un élément fondamental (un fait, une notion, un chiffre, une instruction, un indice…) prélevé à partir d'une expérience. Elle est transformée/traitée par l'être humain ou par des moyens automatiques pour :

- servir à un raisonnement ou à une recherche,
- créer de l'information.

La donnée (sa qualité, sa quantité, sa valeur) peut être déterminée par :

- les intérêts des personnes qui la récoltent et la traitent ;
- les moyens physiques, institutionnels, techniques et méthodologiques choisis pour la prélever ;
- la destination qu'on envisage de lui donner.

2- Qu'est-ce qu'un système d'information ?

Il s'agit de l'ensemble des éléments participant à la gestion, au traitement, au transport et à la diffusion de l'information au sein de l'organisation. Un système d'information est donc constitué d'une infrastructure (matériel), de logiciels, d'intervenants, de procédures et de données.

3- Qu'est qu'une base de données ?

De façon basique, on appelle base de données tout ensemble de données stockées numériquement et pouvant servir à un ou plusieurs programmes. De ce point de vue des fichiers sur un disque dur, un fichier de tableur, voire un fichier de traitement de texte peuvent constituer des bases de données.

D'un point de vue plus restreint, on appellera base de données une collection de données numériques persistantes (durée de vie dépasse celle de l'exécution d'un programme) qui possède une structure ; c'est à dire dont l'organisation répond à une logique systématique, stockées dans des fichiers et accessibles à la demande. On parle de base de données informatisée.

Une base de données est structurée afin de pouvoir mieux répondre à des fonctions fondamentales en informatique, telles que :

- ✓ Stocker l'information de façon fiable (c'est à dire être capable de restituer l'information entrée dans le système);
- ✓ Traiter de grands volumes de données (massification);
- ✓ Traiter rapidement les données (optimisation);
- ✓ Sécuriser les accès aux données (gérer les autorisations selon les utilisateurs);
- ✓ Contrôler la qualité des données (par exemple la cohérence par rapport à un modèle préétabli);
- ✓ Partager les données (entre plusieurs applications dédiées à plusieurs métiers);

Le résultat de la conception d'une base de données informatisée est une description des données. Par description on entend définir les propriétés d'ensembles d'objets modélisés dans la base de données et non pas d'objets particuliers. Les objets particuliers sont créés par des programmes d'applications ou des langages de manipulation lors des insertions et des mises à jour des données.

Cette description des données est réalisée en utilisant un modèle de données. Ce dernier est un outil formel utilisé pour comprendre l'organisation logique des données.

4- Qu'est-ce qu'un système de gestion de base de données ?

Un SGBD est un logiciel qui prend en charge la structuration, le stockage, la mise à jour et la maintenance d'une base de données.

C’est un ensemble de logiciels systèmes permettant aux utilisateurs d'insérer, de modifier, et de rechercher efficacement des données spécifiques dans une grande masse d'informations (pouvant atteindre plusieurs milliards d'octets) partagée par de multiples utilisateurs.

Un SGBD héberge généralement plusieurs bases de données, qui sont destinées à des logiciels ou des thématiques différents.

Actuellement, la plupart des SGBD fonctionnent selon un mode client/serveur. Le serveur (sous-entendu la machine qui stocke les données) reçoit des requêtes de plusieurs clients et ceci de manière concurrente. Le serveur analyse la requête, la traite et retourne le résultat au client.

Le SGBD est l'unique interface entre les informaticiens et les données (définition des schémas, programmation des applications), ainsi qu'entre les utilisateurs et les données (consultation et mise à jour). Cette communication entre les différents acteurs du système est rendu possible à l’aide d’un langage. En pratique, on distingue :

- Le langage de définition des données (LDD) : Il permet de décrire les entités et les associations qui vont constituer les schémas conceptuels, externes et internes de la base. On distingue les commandes CREATE, DROP, ALTER.
- Le langage de manipulation des données (LMD) : Pour décrire les programmes d’applications (consultation, mise à jour des données). On distingue les commandes INSERT, UPDATE, DELETE, SELECT.
- Le langage de contrôle de données (LCD) : Permet le contrôle d’accès aux données et l’intégrité des données. On distingue les commandes COMMIT, ROLLBACK, REVOKE, GRANT.

Un SGBD a pour objectifs :

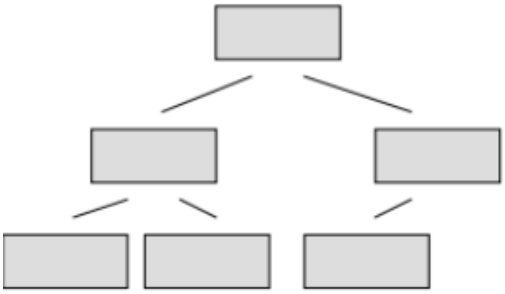
- ✓ la description des données;
- ✓ la recherche et la mise à jour des données ;
- ✓ la sûreté des données (confidentialité) : vérifier les droits d'accès utilisateurs, crypter les informations sensibles;
- ✓ la sécurité des données: sauvegarder et restaurer les données, reprise après panne, journalisation, limiter les erreurs de saisie, de manipulation ;
- ✓ le contrôle de la redondance de l’information ;
- ✓ l'intégrité des données : définir des règles qui maintiennent l'intégrité des données (contraintes d'intégrité);
- ✓ le partage de données : Partager la BD entre plusieurs utilisateurs ;
- ✓ la gestion de la concurrence des accès : détecter et traiter les cas où il y a conflit d'accès entre plusieurs utilisateurs ;

II- Les différents modèles de bases de données

Les bases de données sont apparues à une époque où la nécessité d'un système de gestion de l'information souple se faisait ressentir. Il existe plusieurs modèles de BD, différenciés selon la représentation des données qu'elle contient :

1- Les bases de données hiérarchiques

Dans les SGBD hiérarchiques, les données sont classées hiérarchiquement, selon une arborescence descendante. Ce modèle utilise des pointeurs entre les différents enregistrements. Chaque élément comporte juste un lien vers le niveau inférieur. Les accès aux données commencent par la racine et descendent l'arborescence jusqu'aux détails recherchés.

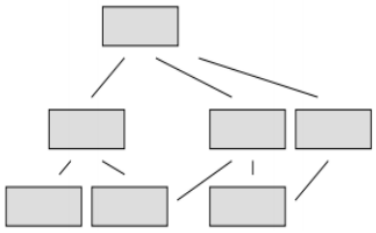


L'accès aux données est certes plus rapide que l'extraction d'un fichier séquentiel, toutefois les performances restent médiocres, puisqu'il faut commencer la recherche à la racine. Une fois la clé accédée, la lecture de l'enregistrement est séquentielle.

Les structures de données hiérarchiques ont été largement utilisées dans les premiers systèmes de gestion de bases de données conçus pour la gestion des données du programme Apollo de la NASA. Cependant, à cause de leurs limitations internes, elles ne peuvent pas souvent être utilisées pour décrire des structures existantes dans le monde réel.

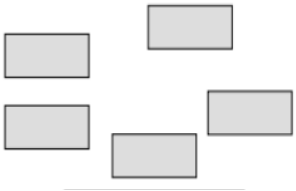
2- Les bases de données réseau

Les bases de données réseau sont des hybrides des modèles relationnel et hiérarchique. Comme le modèle hiérarchique, ce modèle utilise des pointeurs vers des enregistrements. Toutefois la structure n'est plus forcément arborescente dans le sens descendant. La BD comporte de nombreux liens entre les différents éléments de données. L'accès aux données est réalisé par des cheminements divers. Ce modèle de SGBD est peu utilisé.



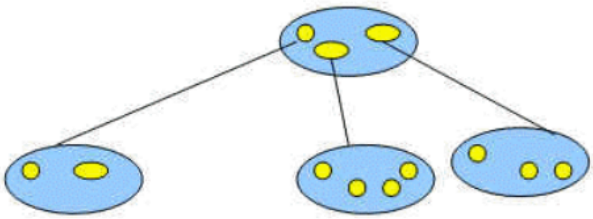
3- Les bases de données relationnelles

Dans les SGBDR (Systèmes de Gestion de Bases de Données Relationnelles), les données sont enregistrées dans des tableaux à deux dimensions (lignes et colonnes). La manipulation de ces données se fait selon la théorie mathématique des relations (opérations ensemblistes). Ce modèle permet une grande indépendance entre les applications, les données et le support physique. Il propose une démarche cohérente et unifiée pour la description et l'interrogation des données tout en supportant le langage SQL qui étend l'algèbre relationnelle.



4- Les bases de données objet

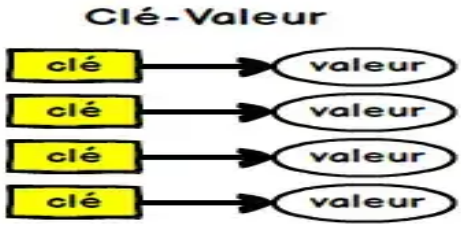
Dans un SGBDO (Système de gestion de bases de données objet) les données sont stockées sous forme d'objets, c'est-à-dire de structures appelées classes présentant des données membres. Les champs sont des instances de ces classes.



5- Les bases de données No-SQL

La base de données NoSQL (Not Only SQL) est **un système de base de données non relationnelle**. Elle est utilisée pour stocker des données sous une forme non structurée, tout ceci

sans toutefois suivre de schémas de construction fixes. Ce système aide à organiser les données de façon libre, sans pour autant respecter la structure contraignante des bases de données relationnelles comme c'est le cas avec le SQL.



Les **bases de données NoSQL** peuvent être utiles dans de nombreux cas d'usage. Elles sont en effet adaptées pour sauvegarder, gérer et analyser de grands volumes de données. Elles sont également utiles lorsque les relations entre les données ne sont pas spécialement importantes.

III- Niveaux d'abstraction des données

La notion d'abstraction des données concerne la représentation générique des données et des associations qui leur donnent un sens. De plus, les traitements essentiels sur le modèle sont pris en compte dès l'étape de la spécification du modèle.

Un objectif majeur des SGBD est d'assurer une abstraction des données stockées sur disques pour simplifier la vision des utilisateurs. Pour cela, trois niveaux de description de données ont été distingués par le groupe ANSI/X3/SPARC. Ces niveaux ne sont pas clairement distingués par tous les SGBD : ils sont mélangés en deux niveaux dans beaucoup de systèmes existants. Cependant, la conception d'une base de données nécessite de considérer et de spécifier ces trois niveaux, certains pouvant être pris en compte par les outils de génie logiciel aidant à la construction des applications autour du SGBD.

1- Niveau externe ou vue

C'est la vue de la BD sous l'angle d'une application particulière. Les données doivent être dans un format compatible avec le langage de programmation, sinon le SGBD doit en assurer la conversion. Le niveau externe assure l'analyse et l'interprétation des requêtes en primitives de plus bas niveau et se charge également de convertir éventuellement les données brutes, issues de la réponse à la requête, dans un format souhaité par l'utilisateur. Le concept de vue permet d'obtenir l'indépendance logique. La modification du schéma logique n'entraîne pas la modification des applications. Chaque vue correspond à la perception d'une partie des données, mais aussi des données qui peuvent être synthétisées à partir des informations représentées dans la BD (par ex. statistiques).

2- Niveau interne ou physique

Le niveau interne correspond à la structure de stockage supportant les données. La définition du schéma interne nécessite au préalable le choix d'un SGBD. Elle permet donc de décrire les données telles qu'elles sont stockées dans la machine, par exemple :

- les fichiers qui les contiennent (nom, organisation, localisation,…) ;
- les articles de ces fichiers (longueur, champs composants,…) ;
- les chemins d'accès à ces articles (index, chaînages,…).

3- Niveau conceptuel ou logique

C'est le niveau central où les structures physiques et les fichiers sont ignorés pour accentuer la description sémantique des données vue par rapport à la réalité d'une organisation. Il correspond à la structure canonique des données qui existent dans l'entreprise, c'est-à-dire leur structure sémantique inhérente sans souci d'implantation en machine, représentant la vue intégrée de tous les utilisateurs. La définition du schéma conceptuel d'une entreprise ou d'une application n'est pas un travail évident. Ceci nécessite un accord sur les concepts de base que modélisent les données. Par exemple, le schéma conceptuel permettra de définir :

- les types de données élémentaires et composés qui permettent de regrouper les attributs ;
- les règles que devront suivre les données au cours de leur vie dans l'entreprise.

Il contient la description des données et des contraintes d'intégrité (Dictionnaire de Données). Le schéma logique découle d'une activité de modélisation.

Chapitre 2 : Introduction à la modélisation conceptuelle : Le modèle entité/association (E/A)

Il est difficile de modéliser un domaine sous une forme directement utilisable par un SGBD. Une ou plusieurs modélisations intermédiaires sont donc utiles, le modèle entités-associations constitue l'une des premières et des plus courantes. Ce modèle permet une description naturelle du monde réel à partir des concepts d'entité et d'association. Basé sur la théorie des ensembles et des relations, ce modèle se veut universel et répond à l'objectif d'indépendance données-programmes. Ce modèle, utilisé pour la phase de conception, s'inscrit notamment dans le cadre d'une méthode plus générale et très répandue : *MERISE*.

MERISE propose une démarche, dite par niveaux, dans laquelle il s'agit de hiérarchiser les préoccupations de modélisation qui sont de trois ordres : la conception, l'organisation et la technique. En effet, pour aborder la modélisation d'un système, il convient de l'analyser en premier lieu de façon globale et de se concentrer sur sa fonction : c'est-à-dire de s'interroger sur ce qu'il fait avant de définir comment il le fait.
Ces niveaux de modélisation sont organisés dans une double approche données/traitements. Les trois niveaux de représentation des données, puisque ce sont eux qui nous intéressent, sont détaillés ci-dessous.

Niveau conceptuel : le *modèle conceptuel des données (MCD)* décrit les entités du monde réel, en termes d'objets, de propriétés et de relations, indépendamment de toute technique d'organisation et d'implantation des données. Ce modèle se concrétise par un *schéma entités-associations* représentant la structure du système d'information, du point de vue des données.

Niveau logique : le *modèle logique des données (MLD)* précise le modèle conceptuel par des choix organisationnels. Il s'agit d'une transcription (également appelée dérivation) du MCD dans un formalisme adapté à une implémentation ultérieure, au niveau physique, sous forme de base de données relationnelle ou réseau, ou autres. Les choix techniques d'implémentation (choix d'un SGBD) ne seront effectués qu'au niveau suivant.

Niveau physique : le *modèle physique des données (MPD)* permet d'établir la manière concrète dont le système sera mis en place (SGBD retenu).

I- Éléments constitutifs du modèle Entité-Association (Modèle E-A)

La *représentation du modèle entités-associations s'appuie sur trois concepts de base :*

- ✓ *L'objet ou entité*
- ✓ *L'association*
- ✓ *La propriété*

L'objet est une entité ayant une existence propre. L'association est un lien ou relation entre objets sans existence propre. La propriété est la plus petite donnée d'information décrivant un objet ou une association.

1- Entité

Définition 1 | Occurrence d'entité : Une occurrence d'entité est un objet, une chose concrète ou abstraite qui peut être reconnue distinctement et qui est caractérisée par son unicité.

Exemples d'occurrences : le comédien Michel GOHOU, l'acteur François TIFFON, le support de cours Pratique SQL, la Toyota qui se trouve au parking, etc.

Les occurrences d'entités ne sont généralement pas représentées graphiquement.

Définition 2 | Entité : Une entité est la représentation d'un objet concret ou abstrait du monde réel, ayant un intérêt pour le domaine étudié.

Les personnes et les livres sont des exemples d'entité.

Remarque : Une entité désigne un ensemble d'occurrences d'entités qui possèdent une sémantique et des propriétés communes.



Figure 2-1 : Représentation graphique d'un exemple d'entité

La figure 2.1 montre la représentation graphique d'un exemple d'*entité* (*Personne*) sans ses propriétés associées.

Les entités *Personne*, caractérisée par un nom et un prénom, et *Voiture*, caractérisée par un nom et une puissance, ne peuvent pas être regroupées, car ils ne partagent pas leurs propriétés (le prénom est une chaîne de caractères et la puissance un nombre). Les entités *Personne*, caractérisée par un nom et un prénom, et *Livre*, caractérisée par un titre et un auteur, possèdent tous les deux, deux attributs du type c*haîne de caractères*. Pourtant, ces deux *entités* ne peuvent pas être regroupées, car ils ne partagent pas une même sémantique : le nom d'une personne n'a rien à voir avec le titre d'un livre, le prénom d'une personne n'a rien à voir avec un auteur.

2- Attribut ou propriété, valeur



Figure 2-2 : Représentation graphique d'un exemple d'entité comportant trois attributs

Définition 3 | attribut, propriété : Un attribut (ou une propriété) est une caractéristique associée à une entité ou à une association.

Exemples d'attribut : le nom d'une personne, le titre d'une livre, la puissance d'une voiture.

Définition 4 | valeur : Au niveau de l'entité ou de l'association, chaque attribut possède un domaine qui définit l'ensemble des valeurs possibles qui peuvent être choisies pour lui (entier, chaîne de caractères, booléen…). Au niveau de l'occurrence, chaque attribut possède une valeur compatible avec son domaine.

La figure 2.2 montre la représentation graphique d'un exemple d'entité (*Personne*) avec trois attributs.

Règle 1 : Un attribut ne peut en aucun cas être partagé par plusieurs entités ou associations.

Règle 2 : Un attribut est une donnée élémentaire, ce qui exclut des données calculées ou dérivées.

Règle 3 : Une entité et ses attributs doivent être cohérents entre eux (*i.e.* ne traiter que d'un seul sujet).

Par exemple, si le modèle doit comporter des informations relatives à des articles et à leur fournisseur, ces informations ne doivent pas coexister au sein d'une même *entité*. Il est préférable de mettre les informations relatives aux articles dans une entité *Article* et les informations relatives aux fournisseurs dans une entité *Fournisseur*. Ces deux entités seront probablement ensuite reliées par une *association*.

3- Identifiant ou clé

Définition 5 : | Identifiant, clé : Un identifiant (ou clé) d'une entité ou d'une association est constitué par un ou plusieurs de ses attributs qui doivent avoir une valeur unique pour chaque occurrence d'entité ou d'association.

Il est donc impossible que les attributs constituant l'identifiant d'une entité (respectivement association) prennent la même valeur pour deux occurrences d'entités (respectivement deux occurrences d'associations) distinctes.

Exemples d'identifiant : le numéro de sécurité sociale pour une personne, le numéro d'immatriculation pour une voiture, le code ISBN d'un livre pour un livre (mais pas pour un exemplaire).

Règle 4 : Chaque entité possède au moins un identifiant, éventuellement formé de plusieurs attributs.

Ainsi, chaque entité possède au moins un attribut qui, s'il est seul, est donc forcément l'identifiant.

Dans la représentation graphique, les attributs qui constituent l'identifiant sont soulignés et placés en tête.

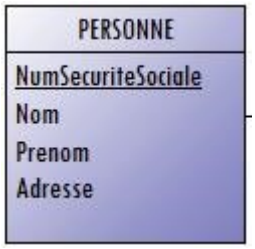


Figure 2.3 : Représentation graphique d'un exemple d'entité comportant quatre attributs dont un est un identifiant : deux personnes peuvent avoir le même nom, le même prénom et la même adresse, mais pas le même numéro de sécurité sociale.

Pour une entité, un identifiant doit respecter les caractéristiques suivantes :

- On choisit un attribut ou groupe d'attributs qui permet de repérer une occurrence de manière unique.
- On souligne l'identifiant.
- On privilégie l'identifiant le plus court, le plus naturel.
- On peut créer un identifiant artificiel.
- L'identifiant ne doit pas varier dans le temps.

Pour les associations, l'identifiant est implicite et obtenu en juxtaposant les identifiants des entités qui participent à la liaison. Toutefois, On peut créer un identifiant artificiel.

4- Association

Définition 6 : | Occurrence d'association Une occurrence d'association est un lien entre plusieurs occurrences d'entités.

Exemples d'occurrence d'association : l'emprunt par l'étudiant Yawo du 3e exemplaire du livre « Maîtrisez SQL ».

Les occurrences d'associations ne sont généralement pas représentées graphiquement.

Définition 7 : | Association Une association désigne un ensemble de liens qui possèdent les mêmes caractéristiques.

Une association décrit un lien entre plusieurs entités. Les occurrences de cette association lient des occurrences de ces entités.

Elle se représente par le biais d'un ovale ou d'un losange contenant son nom et ses éventuels attributs.

Comme les entités, les associations sont définies à l'aide d'attributs qui prennent leur valeur dans les occurrences d'associations.

Règle 5 : Un attribut peut être placé dans une association uniquement lorsqu'il dépend de toutes les occurrences d'entités liées par l'association.

Une association peut ne pas posséder d'attribut explicite et cela est relativement fréquent, mais on verra qu'il possède au moins des attributs implicites.

Exemples d'*association* : l'emprunt d'un livre à la bibliothèque.

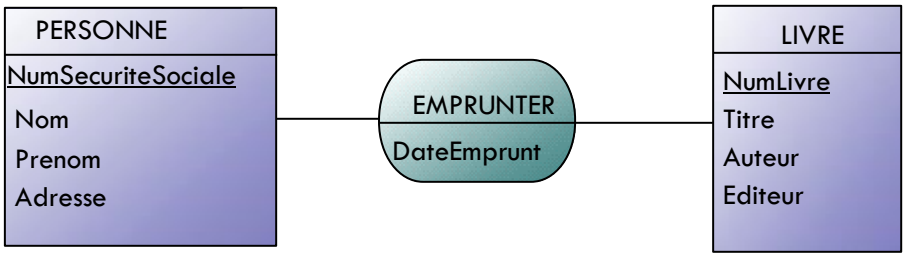


Figure 2.4 : Représentation graphique d'un exemple d'association liant deux entités.

5- Cardinalité

Définition 8 : | Cardinalité La cardinalité d'une patte reliant une association et une entité précise le nombre de fois minimal et maximal d'interventions d'une occurrence de l'entité dans une occurrence de l'association. La cardinalité minimale doit être inférieure ou égale à la cardinalité maximale.

Exemple de cardinalité : une personne peut être l'auteur de 0 à *n* livre, mais un livre ne peut être écrit que par une personne.

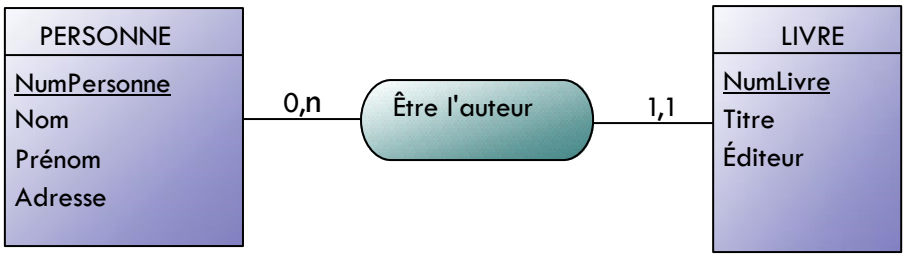


Figure 2.5 : Représentation graphique des cardinalités d'une association. Dans cet exemple pédagogique, on suppose qu'un livre ne peut posséder qu'un auteur.

Règle 6 : L'expression de la cardinalité est obligatoire pour chaque patte d'une association.

Ainsi, si une cardinalité maximale est connue et vaut 2, 3 ou plus, alors nous considérons qu'elle est indéterminée et vaut *n*. En effet, si nous connaissons *n* au moment de la conception, il se peut que cette valeur évolue au cours du temps. Il vaut donc mieux considérer *n* comme inconnue dès le départ. De la même manière, on ne modélise pas des cardinalités minimales qui valent plus de 1, car ces valeurs sont également susceptibles d'évoluer. Enfin, une cardinalité maximale de 0 n'a pas de sens, car elle rendrait l'association inutile.

Les seules cardinalités admises sont donc :

0,1 : une occurrence de l'*entité* peut exister tout en n'étant impliquée dans aucune occurrence de l'association et peut être impliquée dans au maximum une occurrence de l'association.

0, n : c'est la cardinalité la plus ouverte ; une occurrence de l'*entité* peut exister tout en n'étant impliquée dans aucune occurrence de l'association et peut être impliquée, sans limitation, dans plusieurs occurrences de l'association.

1,1 : une occurrence de l'*entité* ne peut exister que si elle est impliquée dans exactement (au moins et au plus) une occurrence de l'association.

1, n : une occurrence de l'*entité* ne peut exister que si elle est impliquée dans au moins une occurrence de l'association.

Une cardinalité minimale de 1 doit se justifier par le fait que les occurrences de l'entité en question ont besoin de l'occurrence de l'association pour exister. Dans tous les autres cas, la cardinalité minimale vaut 0.

II- Types d'associations

1- Associations plurielles

Définition 9 :| Associations plurielles Deux entités peuvent être liées par plusieurs relations : on parle d'associations plurielles.

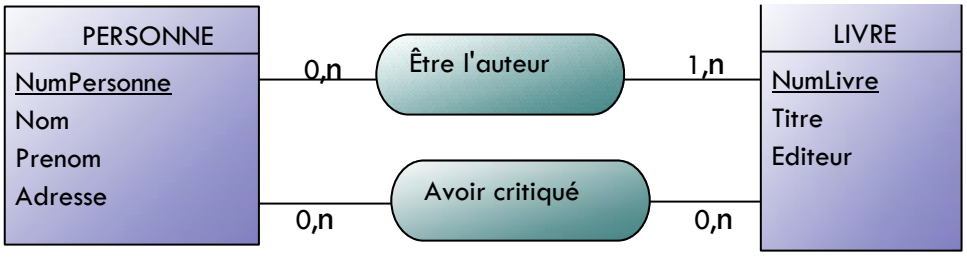


Figure 2.6 : Exemple d'associations plurielles entre une entité Personne et une entité Livre. Sur ce schéma, une association permet de modéliser que des personnes écrivent des livres et une autre que des personnes critiquent (au sens de critique littéraire)

2- Association réflexive

Définition 10 :| Association réflexive Une association est qualifiée de réflexive quand elle matérialise une relation entre une entité et elle-même (cf. figure 2.7).

Une occurrence de cette association associe généralement une occurrence de l'association à une autre entité du même type. Cette relation peut être symétrique, c'est le cas de l'association *Être frère* sur la figure 2.7, ou ne pas l'être, comme l'association *Être parent* sur cette même figure. Dans le cas où la relation n'est pas symétrique, on peut préciser les rôles sur les pattes de l'association comme pour la relation Être parent de la figure 2.7. L'ambiguïté posée par la non-symétrie d'une association réflexive sera levée lors du passage au modèle relationnel.

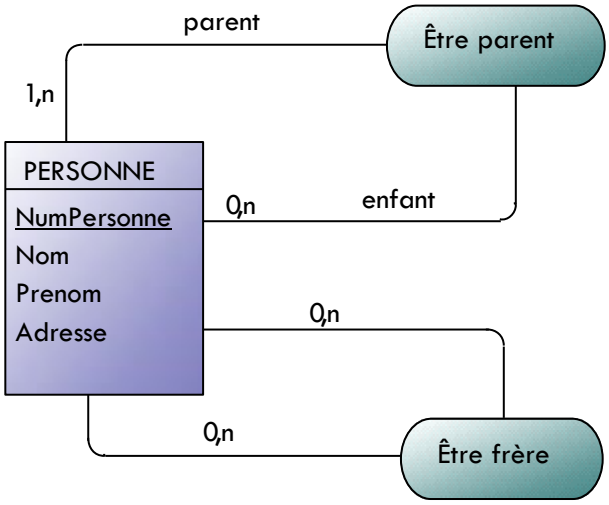


Figure 2.7 : Exemple d'associations réflexives sur l'entité Personne. La première association permet de modéliser la relation parent/enfant et la deuxième association la relation de fraternité.

3- Association n-aire (n>2)

Définition 11 :| Association n-aire Une association n-aire est une association qui met en relation n entités. Même s'il n'y a, en principe, pas de limite sur le nombre d'entités qui participent à l'association dans la pratique on va rarement au-delà de trois. Les associations de degré supérieur à deux sont plus difficiles à manipuler et à interpréter, notamment au niveau des cardinalités.

La solution consiste à éclater l'association ternaire en des associations binaires comme représenté sur la figure 2.9.

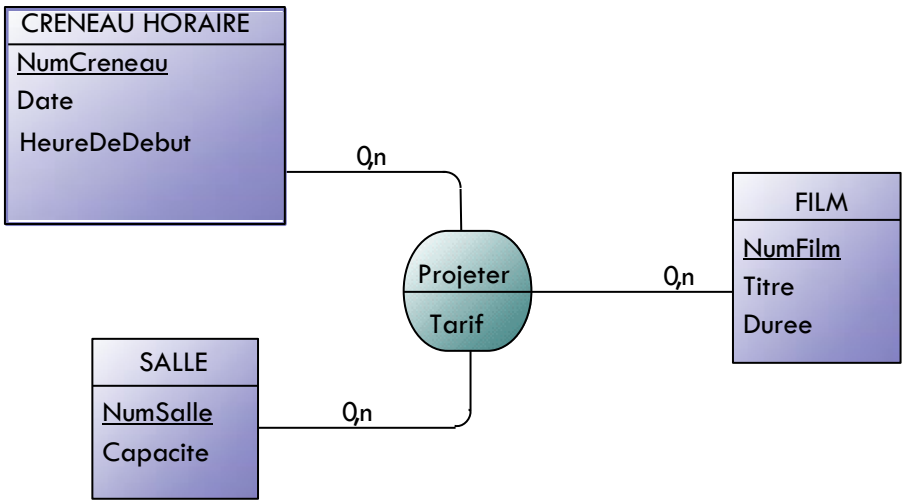


Figure 2.8 : Exemple d'association ternaire entre les entités Créneau horaire, Salle et Film.

La figure 2.8 nous montre un exemple d'association ternaire entre les entités *Créneau horaire*, *Salle* et *Film*. Il est toujours possible de s'affranchir d'une association n-aire (n>2) en se ramenant à des associations binaires de la manière suivante :

- ✓ On remplace l'association n-aire par une entité et on lui attribue un identifiant.
- ✓ On crée des associations binaires entre la nouvelle entité et toutes les entités de la collection de l'ancienne ***association*** n-aire.

- ✓ La cardinalité de chacune des associations binaires créées est 1,1 du côté de l'*entité* créée (celle qui remplace l'association n-aire), et 0, *n* ou 1, *n* du côté des entités de la collection de l'ancienne association n-aire.

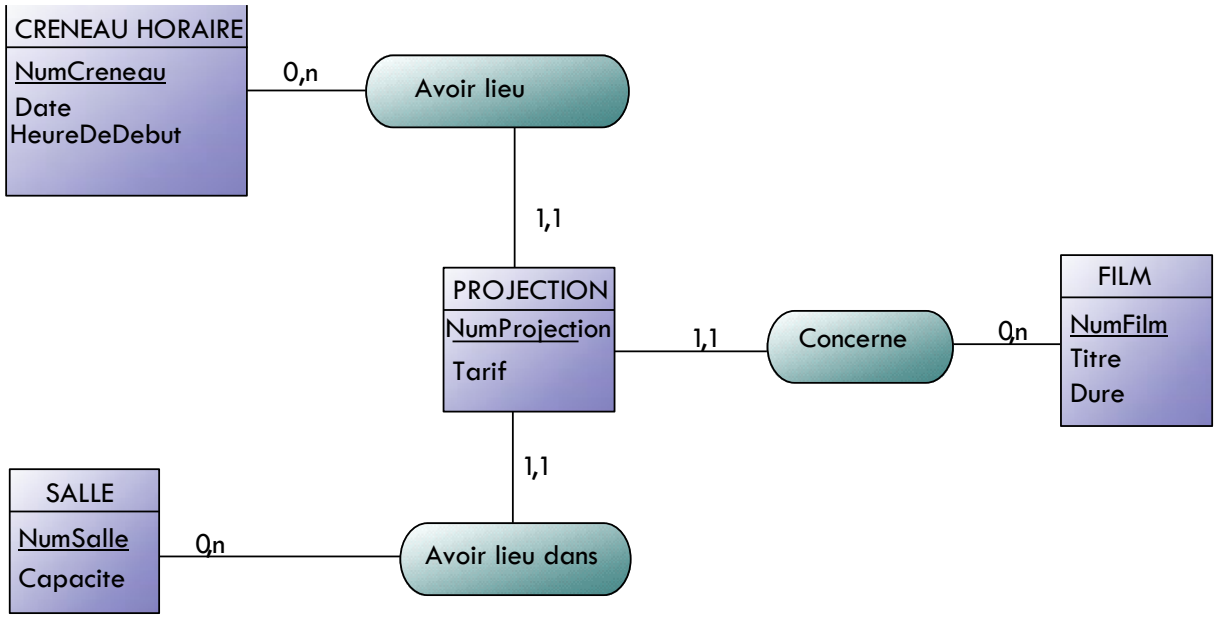


Figure 2.9 : Transformation de l'association ternaire de la figure 2.10 en une entité et trois associations binaires.

La figure 2.9 illustre le résultat de cette transformation sur le schéma de la figure 2.8.

L'avantage du schéma de la figure 2.9 est de rendre plus intelligible la lecture des cardinalités. Ainsi, le mécanisme, que nous venons de détailler ci-dessus, de passage d'une association n-aire (n>2) à une entité et n associations binaires est tout à fait réversible à condition que :

- ✓ Toutes les pattes des associations binaires autour de l'entité centrale ont une cardinalité maximale de 1 au centre et de n à l'extérieur ;
- ✓ Les attributs de l'entité centrale satisfont la règle de bonne formation des attributs de l'association.

Chapitre 3 : Le Modèle relationnel

Le modèle relationnel a été introduit en 1970 par Edgar Frank Codd, informaticien britannique travaillant à IBM. La première volonté du modèle relationnel fut d'être un modèle ensembliste simple, supportant des ensembles d'enregistrements aussi bien au niveau de la description que de la manipulation. Le modèle relationnel est donc basé sur la théorie mathématique des relations sachant que le concept de relation découle directement de la théorie des ensembles. Il est aujourd'hui la base de nombreux systèmes, et les architectures permettant d'accéder depuis une station de travail à des serveurs de données s'appuient en général sur lui. Le relationnel a donc atteint ses objectifs au-delà de toute espérance.

I- Présentation

Les données sont représentées par des tables, sans préjuger de la façon dont les informations sont stockées dans la machine. Les tables constituent donc la structure logique du modèle relationnel. Au niveau physique, le système est libre d'utiliser n'importe quelle technique de stockage (fichiers séquentiels, indexage, adressage dispersé, séries de pointeurs, compression…) dès lors qu'il est possible de relier ces structures à des tables au niveau logique. Les tables ne représentent donc qu'une abstraction de l'enregistrement physique des données en mémoire.

Le succès du modèle relationnel auprès des chercheurs, concepteurs et utilisateurs est dû à la puissance et à la simplicité de ses concepts. En outre, contrairement à certains autres modèles, il repose sur des bases théoriques solides, notamment la théorie des ensembles et la logique des prédicats du premier ordre.

Les objectifs du modèle relationnel sont :

- Proposer des schémas de données faciles à utiliser ;
- Améliorer l'indépendance logique et physique ;
- Mettre à la disposition des utilisateurs des langages de haut niveau ;
- Optimiser les accès à la base de données ;
- Améliorer l'intégrité et la confidentialité ;
- Fournir une approche méthodologique dans la construction des schémas.

De façon informelle, on peut définir le modèle relationnel de la manière suivante :

- Les données sont organisées sous forme de tables à deux dimensions, encore appelées relations, dont les lignes sont appelées n-uplet ou tuple en anglais ;
- Les données sont manipulées par des opérateurs de l'algèbre relationnelle ;
- L'état cohérent de la base est défini par un ensemble de contraintes d'intégrité.

Au modèle relationnel est associée la théorie de la normalisation des relations qui permet de se débarrasser des incohérences au moment de la conception d'une base de données relationnelle.

II- Éléments du modèle relationnel

Domaine : c'est un ensemble, fini ou infini, de valeurs, caractérisé par un nom.

Exemple : entier, alphabet, couleur-de-vin.

NB : Il est possible de définir les domaines en intention ou par extension.

Produit cartésien : le produit cartésien d'un ensemble de domaines D_1, D_2, ..., D_n est l'ensemble des vecteurs $<v_1, v_2..., v_n>$ où, pour i variant de 1 à n, v_i est une valeur de D_i.

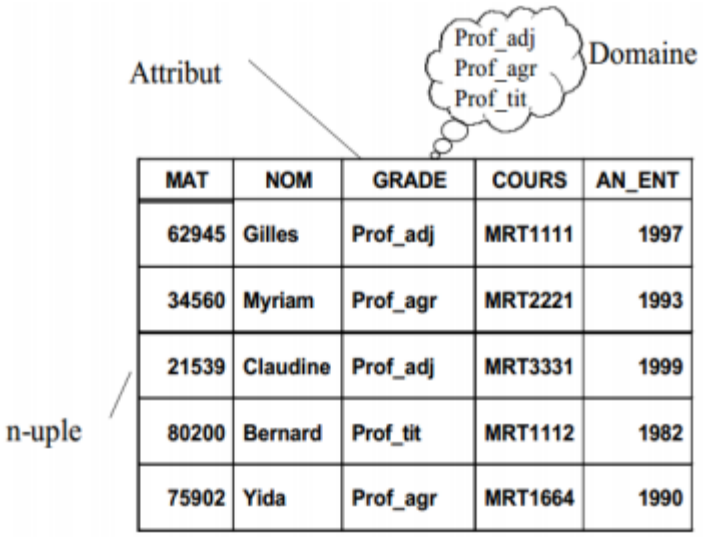
Ex : Le produit cartésien des domaines ETUDIANT = {Kodjo, Toto, Jean} et MATIERE = {Algèbre, ATO} est composé de six vecteurs représentés ci-dessous.

Kodjo	Algèbre
Kodjo	ATO
Toto	Algèbre
Toto	ATO
Jean	Algèbre
Jean	ATO

Relation : c'est un sous-ensemble du produit cartésien d'une liste de domaines caractérisé par un nom. $R \subseteq D_1 \times D_2 \times \ldots \times D_n$

Représentation

Une relation se présente sous la forme d'un tableau à deux dimensions dont les colonnes correspondent aux domaines et les lignes sont les n-uplets (tuples).



MAT	NOM	GRADE	COURS	AN_ENT
62945	Gilles	Prof_adj	MRT1111	1997
34560	Myriam	Prof_agr	MRT2221	1993
21539	Claudine	Prof_adj	MRT3331	1999
80200	Bernard	Prof_tit	MRT1112	1982
75902	Yida	Prof_agr	MRT1664	1990

N-uplet (tuple) : C'est une ligne d'une relation (ou table) correspondant à un enregistrement.

Attribut : C'est une colonne d'une relation caractérisée par un nom. Il explicite le rôle joué par un domaine dans une relation.

Autres concepts et règles :

- Il n'existe pas deux n-uplets identiques.
- L'ordre d'apparition des n-uplets est quelconque.
- *Cardinalité d'une relation* : c'est le nombre de n-uplets contenus dans la relation.
- Le nombre de domaines caractérise une relation. Une relation à n domaines est dite de degré n.
- *Clé d'une relation* : c'est un sous-ensemble d'attributs qui permet d'identifier sans ambiguïté un n-uplet de la relation.
- *Schéma de relation* : c'est la représentation d'une relation en intention *(par opposition à une table qui la définit par extension).*

  Exemple : Étudiant (matricule, nom, prénom, dateNaiss, sexe, nationalité)

- Le schéma conceptuel d'une base de données est constitué de l'ensemble des schémas de relation à implémenter.
- La base de données est l'ensemble des tables implémentant les schémas de relation.
- *Domaine primaire* : C'est un domaine sur lequel est définie une clé primaire (donc mono-attribut)

- *Clé étrangère* : un attribut qui n'est pas clé primaire mais qui est défini sur un domaine primaire dans une autre relation est dit clé étrangère. Elle exprime les associations entre entités.

III- Passage du modèle entités-associations au modèle relationnel

Pour traduire un schéma du modèle entités-associations vers le modèle relationnel, on peut appliquer les règles suivantes :

1. Chaque **entité** donne naissance à une **relation**. Chaque attribut de cette entité devient un attribut de la relation. **L'identifiant** est conservé en tant que **clé** de la relation.

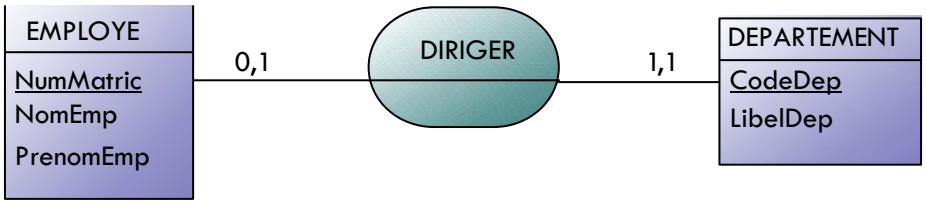
2. Chaque **association** dont aucune patte n'a pour cardinalité maximale 1 donne naissance à une **relation**. Chaque attribut de cette association devient un attribut de la relation. L'identifiant, s'il est précisé, est conservé en tant que clé de la relation, sinon **cette clé est formée par la concaténation des identifiants** des entités qui interviennent dans l'association.

3. Une association dont au moins une patte a une cardinalité maximale à 1 (cette association devrait être binaire et n'a généralement pas d'attribut) ne devient pas une relation. Elle décrit en effet une **contrainte d'intégrité fonctionnelle***. La relation correspondant à l'entité dont la patte vers l'association a une cardinalité maximale valant 1, se voit simplement ajouter comme attribut (et donc comme clé étrangère) l'identifiant de l'autre entité.

* Contrainte d'intégrité fonctionnelle (en abrégé CIF) se définit par le fait que l'une des entités participant à l'association est complètement déterminée par la connaissance d'une ou plusieurs autres entités participant dans cette même association.

Exemple : Entre Département et Employé (Travailler), il y a une CIF. La connaissance d'un employé permet de remonter sans erreur au département dans lequel il travaille.

1- Cas particulier d'une association du type 1 vers 1



Dans l'exemple ci-dessus, toutes les cardinalités maximales de l'association *Diriger* sont de 1. L'application des règles de passage du modèle entités-associations au modèle relationnel énoncées plus haut nous donnerait :

- EMPLOYE (NumMatric, NomEmp, PrenomEmp, #CodeDep)
- DEPARTEMENT (CodeDep, LibelDep, #NumMatric)

L'attribut CodeDep dans la relation Employé est une clé étrangère de la relation Département. De même, l'attribut NumMatric dans la relation Département est une clé étrangère de la relation Employé.

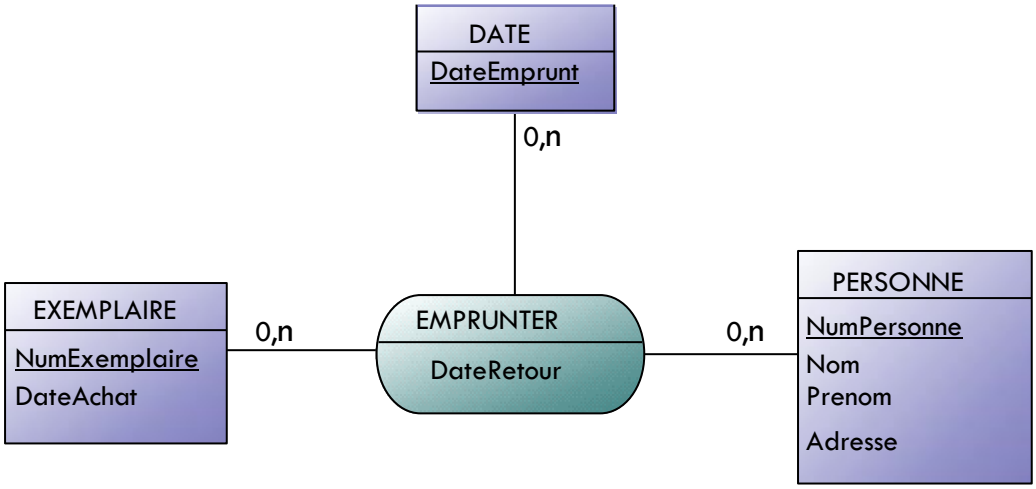
L'association *Diriger* étant du type 1 vers 1, elle est entièrement matérialisée dans la relation Département par l'attribut NumMatric. Il est donc inutile de la rematérialiser dans la relation Employé. L'attribut CodeDep dans la relation Employé doit donc être supprimé.

Le schéma relationnel adéquat correspondant au modèle entités-associations ci-dessus devient donc :

- EMPLOYE (NumMatric, NomEmp, PrenomEmp)
- DEPARTEMENT (CodeDep, LibelDep, #NumMatric)

2- Cas particulier d'une entité sans attribut autre que sa clé

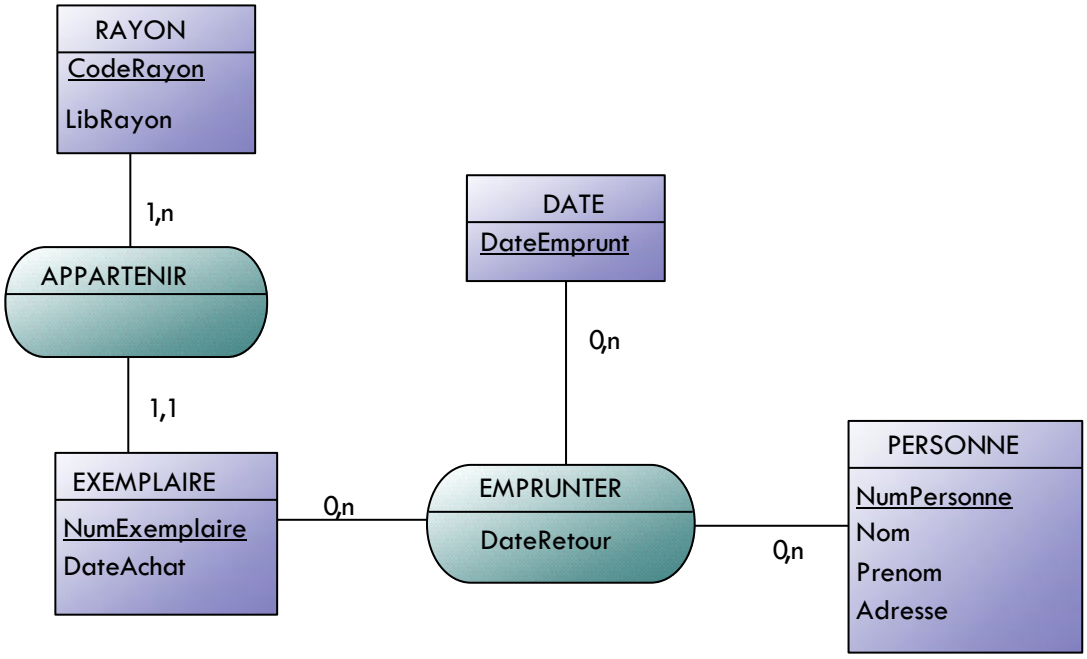
Lorsqu'une entité ne possède pas d'attribut en dehors de sa clé, il ne faut pas nécessairement en faire une relation.



Par exemple, l'entité Date de la figure ci-dessus ne doit pas se traduire par une relation. Le schéma relationnel adéquat correspondant au modèle entités-associations de la figure est donc :

- EXEMPLAIRE (NumExemplaire, DateAchat)
- PERSONNE (NumPersonne, Nom, Prenom, Adresse)
- EMPRUNTER (#NumExemplaire, #NumPersonne, DateEmprunt, DateRetour)

3- Exemple complet



Comme exemple d'application, voici les relations déduites du schéma entités- associations de la figure ci-dessus :

- RAYON (CodeRayon, LibRayon)
- EXEMPLAIRE (NumExemplaire, DateAchat, #CodeRayon)
- PERSONNE (NumPersonne, Nom, Prenom, Adresse)
- EMPRUNTER (#NumExemplaire, #NumPersonne, DateEmprunt, DateRetour)

Chapitre 4 : Manipulation des données : Algèbre relationnelle

I- Introduction

La représentation d'information sous forme relationnelle est intéressante car les fondements mathématiques du relationnel, outre qu'ils permettent une modélisation logique simple et puissante, fournissent également un ensemble de concepts pour manipuler formellement l'information ainsi modélisée.

Il s'agit d'un ensemble d'opérateurs qui prennent en entrée des relations et qui produisent en sortie des relations.

- **Objectif : Localiser des données dans la base qui répondent à un certain critère. Permet de faire des recherches dans les relations.**

Ainsi une algèbre relationnelle, sous forme d'un ensemble d'opérations formelles, permet d'exprimer des questions, ou requêtes, posées à une représentation relationnelle, sous forme d'expressions algébriques.

L'algèbre relationnelle est principalement composée par les cinq opérateurs de base et les trois opérateurs additionnels suivants :

Opérateurs de base

- Projection
- Restriction
- Produit cartésien
- Union
- Différence

Opérateurs additionnels

- Intersection
- Jointure
- Division

II- Les opérateurs

1- La projection

La projection de R1 sur une partie de ses attributs {A1, A2, ...} produit une relation R dont le schéma est restreint aux attributs mentionnés en opérande, comportant les mêmes tuples que R1, et dont les doublons sont éliminés.

La projection est une opération unaire.

Notation: R = PROJECTION (R1, A1, A2, … An) / R = $\pi_{[attributs]}$ (R1).

Exemple : Soit la relation suivante Personne.

id	nom	prenom	num_tel	sexe	age
1	AMAH	Eric	90645794	M	18
2	HOUTSHA	Nandine	98230987	F	25
3	ASSIH	Francine	99098745	M	35
4	GOMA	Emmanuel	90000102	F	41

Soit l'opération : R = Projection (Personne, nom, prenom) ou R = π nom, prenom (Personne).

On obtient alors la relation R composée des tuples suivants :

nom	prenom
AMAH	Eric
HOUTSHA	Nandine
ASSIH	Francine
GOMA	Emmanuel

2- Restriction

La restriction de R1, étant donnée une condition C, produit une relation R de même schéma que R1 et dont les tuples sont les tuples de R1 vérifiant la condition C de type < prédicat-élémentaire [opérateur -logique prédicat-élémentaire] >.

- ✓ opérateur-logique {et, ou}
- ✓ prédicat-élémentaire : attribut opérateur-de-comparaison constante|attribut (opérateur-decomparaison {=, ≠,<,>, ≤, ≥})

La restriction est une opération unaire.

Notation : R = Restriction (R1, condition) / R = $\sigma_{[condition]}$ (R1)

Exemple : Soit la relation suivante Personne.

id	nom	prenom	num_tel	sexe	age
1	AMAH	Eric	90645794	M	18
2	HOUTSHA	Nandine	98230987	F	25
3	ASSIH	Francine	99098745	M	35
4	GOMA	Emmanuel	90000102	F	41

Soit l'opération : R = Restriction (Personne, sexe= 'F')

On obtient alors la relation R composée des tuples suivants :

id	nom	prenom	num_tel	sexe	age
2	HOUTSHA	Nandine	98230987	F	15
4	GOMA	Emmanuel	90000102	F	41

Soit l'opération : R = σ sexe= 'M'^age>20 (Personne).

On obtient alors la relation R composée du tuple unique suivant :

id	nom	prenom	num_tel	sexe	age
3	ASSIH	Francine	99098745	M	35

3- Produit cartésien

Le produit cartésien de R1 par R2 (équivalent au produit de R2 par R1) produit une relation R ayant pour schéma la juxtaposition de ceux des relations R1 et R2 et pour tuples l'ensemble des combinaisons possibles entre les tuples de R1 et ceux de R2.
Le produit cartésien est une opération binaire. Il est commutatif et associatif.

- Le nombre de colonne du produit de R1 par R2 est égal au nombre de colonne de R1 plus le nombre de colonnes de R2. ***Degré R = Degré R1 + Degré R2***
- Le nombre de tuples résultant du produit de R1 par R2 est égal au nombre de tuples de R1 fois le nombre de tuples de R2. ***Cardinalité R = Cardinalité R1 * Cardinalité R2***

Notation : R = Produit (R1, R2) / R = R1 × R2

Exemple : Soit les relations R1 et R2 suivantes.
R1

A	B	C
a1	b1	c1
a2	b2	c2
a3	b3	c3

R2

A	D
a1	d1
a2	d2

Soit l'opération : R = Produit (R1, R2)

On obtient alors la relation R composée des tuples suivants :

R1.A	B	C	R2.A	D
a1	b1	c1	a1	d1
a1	b1	c1	a2	d2
a2	b2	c2	a1	d1
a2	b2	c2	a2	d2
a3	b3	c3	a1	d1
a3	b3	c3	a2	d2

4- Jointure

La jointure de R1 et R2, étant donné une condition C portant sur des attributs de R1 et de R2, de même domaine, produit une relation R ayant pour schéma la juxtaposition de ceux des

relations R1 et R2 et pour tuples l'ensemble de ceux obtenus par concaténation des tuples de R1 et de R2, et qui vérifient la condition C.
Elle est une opération qui entraîne la perte de certains tuples : ceux qui appartiennent à une des deux relations opérandes et qui n'ont pas de correspondance dans l'autre relation.

La jointure est une opération binaire. Elle est commutative et associative.

Notation : R = Jointure (R1, R2, condition)

Exemple : Soit les deux relations suivantes :

Personne

id	nom	prenom	num_tel	sexe	age
1	AMAH	Eric	90645794	M	18
2	HOUTSHA	Nandine	98230987	F	25
3	ASSIH	Francine	99098745	M	35
4	GOMA	Emmanuel	90000102	F	41

Voiture

immatriculation	type	marque	proprietaire
TG-9854-AK	corolla	toyota	2
TG-6540-AM	anaconda	bmw	1
TG-0923-AO	yaris	toyota	4
TG-1755-AP	c300	benz	2

Soit l'opération suivante: R = Jointure (Personne, Voiture, Personne.id=Voiture.propriétaire)

On obtient alors la relation R composée des tuples suivants :

id	nom	prenom	num_tel	sexe	age	**immatriculation**	type	marque	proprietaire
1	AMAH	Eric	90645794	M	18	TG-6540-AM	anaconda	bmw	1
2	HOUTSHA	Nandine	98230987	F	25	TG-9854-AK	corolla	toyota	2
2	HOUTSHA	Nandine	98230987	F	25	TG-1755-AP	c300	benz	2
4	GOMA	Emmanuel	90000102	F	41	TG-0923-AO	yaris	toyota	4

Chapitre 5 : Introduction au langage SQL

I- Présentation du Langage SQL

Le module précédent, conception des bases de données, a détaillé le côté théorique et mathématique de la création, de l'actualisation et de la récupération de tables dans les bases de données relationnelles. Ce chapitre présente une introduction générale sur la façon dont se réalisent ces tâches dans des SGBDR particuliers. Nous ne présenterons qu'un aperçu de SQL. Pour des informations plus détaillées sur les capacités de ce langage, consultez d'autres sources qui sont plus orientées sur le langage SQL.

SQL est le langage informatique standard pour la communication avec les systèmes de gestion de bases de données relationnelles. Le standard SQL a été défini par **l'ANSI** (*American National Standard Institute*) et **l'ISO** (*International Standard Organisation*). Le nom officiel du langage est International Standard Database Langage SQL (1992). La dernière version de ce langage est appelée en général SQL/92 ou SQL2. Dans la suite du cours faisons référence à ce standard par les termes standard **SQL ANSI/ISO** ou plus simplement, standard **SQL**.

Oracle Corporation a réalisé la première implémentation commerciale du langage en 1979. Bien que la plupart des éditeurs de bases de données relationnelles prennent en charge SQL/92, la conformité au standard n'est pas totale. Actuellement, il existe plusieurs variantes de SQL sur le marché parce que chaque éditeur de SGBDR tente d'étendre le standard pour accroître l'attrait commercial de son produit.

Dans ce module de pratique SQL, nous illustrons les implémentations des fonctionnalités de SQL avec le SGBDR ***MySQL***. Il est important de remarquer que de nombreux éditeurs de SGBDR fournissent des moyens interactifs pour créer et mettre à jour des bases de données au moyen de tables et/ou fenêtres.

Bien que vous verrez d'autres moyens d'actualiser des tables et de créer des requêtes, vous devez vous rappeler que SQL est toujours le langage en arrière-plan mise en œuvre par MySQL.

1- Caractéristiques de SQL

L'une des principales caractéristiques du langage SQL est qu'il s'agit d'un langage ***déclaratif*** ou ***non procédural***.

Le programmeur n'a donc pas à se spécifier, étape par étape toutes les opérations que l'ordinateur doit exécuter pour obtenir un résultat particulier ; in indique au système de gestion de bases de données ce qui doit être accompli et il laisse alors le système décider par lui-même comment obtenir le résultat indiqué.

2- Sous-langages de données de SQL

Les instructions ou commandes du langage SQL se divisent généralement en ***trois*** catégories principales ou sous-langages de données déjà expliqué dans le module précédent à savoir :

- ***Le langage de définition des données*** (**LDD** pour **DDL** en anglais) : Le DDL inclut les instructions qui prennent en charge la définition ou la création d'objets de la base de données tels que les tables, les index, des séquences et des vues. Parmi les instructions DDL les plus généralement rencontrées, on trouve les différentes formes des commandes ***CREATE, ALTER, DROP***.
- ***Le langage de manipulation des données*** (**LMD** pour **DML** en anglais) : DML comprend les instructions qui permettent le traitement ou la manipulation d'objet de base de données. Parmi les instructions DML les plus utilisées, on trouve les différentes modalités de ***SELECT, INSERT, DELETE et UPDATE***.
- ***Le langage de contrôle de données*** (**LCD**) : Permet le contrôle d'accès aux données et l'intégrité des données. On distingue les commandes COMMIT, ROLLBACK, ***REVOKE, GRANT***.

Il est important d'observer que tous les objets crées dans une base de données sont stockés dans le dictionnaire ou catalogue de la base de données.

3- SQL interactif ou sous sa forme intégrée ?

Le langage SQL peut s'employer de façon ***interactive*** ou sous sa ***forme intégrée***.

SQL interactif vous permet d'exécuter des commandes directement sur le SGBDR et de recevoir les résultats immédiatement.

Lorsque le ***SQL intégré*** est implémenté, les instructions SQL font partie d'un programme écrit dans un langage général tel que *JAVA*, *PHP* ou C++. Dans ce cas, le langage de programmation général est désigné comme étant le langage ***hôte***.

Le principal intérêt de SQL intégré est de disposer des fonctionnalités d'un langage de programmation additionnel qui ne sont pas prises en charge par SQL.

Lorsqu'il fait appel à SQL intégré, l'utilisateur n'observe pas directement la sortie des différentes instructions SQL. En effet, les résultats sont transmis à des variables ou à des paramètres de procédures.

En règle générale, toute instruction SQL qui peut s'utiliser de façon interactive peut aussi être intégrée dans un programme d'application. Toutefois, l'utilisateur doit garder à l'esprit qu'il peut y avoir certaines différences syntaxiques dans les instructions SQL, selon qu'elles sont appelées de façon interactive ou intégrées dans un programme. Dans notre contexte, nous ne considérons SQL que sous sa forme interactive.

II- Structure des instructions SQL

Toute instruction ou commande SQL est une combinaison d'une ou de plusieurs ***clauses***. Les *clauses* sont, en général, introduites par des mots clés.

Un exemple d'instruction SQL est donnée figure III.1. L'administrateur de base de données n'est pas concerné par le fonctionnement interne de cette instruction mais seulement par ses aspects structurels.

```
SELECT nom_colonne_1, nom_colonne_2, ..., nom_colonne_n
FROM nom_table
WHERE condition
ORDER BY nom_colonne_1 [ASC | DESC] [, nom_colonne_2 [ASC | DESC], ...] ;
```

figure III.1 Mots clés et clauses dans la structure d'une instructions SQL

Dans cette instruction SQL, nous distinguons *quatre mots-clés et quatre clauses*.

Les mots clés sont en gras sur la *figure III.1.* Comme indiqué précédemment, un mot clé est un terme qui a une signification particulière dans un langage. Faire appel à un mot-clé ailleurs que dans son contexte spécifique génère des erreurs.

Les quatre clauses de l'instruction SQL de la *figure III.1* débutent par un mot clé.

Dans l'instruction précédente, les deux premières clauses sont obligatoires **(SELECT et FROM)** et les deux dernières **(WHERE et ORDER BY)** sont facultatives.

Lorsque nous décrivons la syntaxe des instructions SQL, nous indiquons les mots clés ou les clauses facultatives en les plaçant entre crochets "**[ ]**". Avec cette instruction, nous pouvons réécrire l'instruction précédente comme montré en *figure III.2.*

Remarquez que dans la clause ORDER BY nous avons placé entre crochets les mots ***ASC*** (ascendant) et ***DESC*** (descendant) et en les séparant par un caractère | ; ce caractère parfois appelé "*pipe*" sert à distinguer les différentes options dont dispose un utilisateur lorsqu'il écrit une instruction SQL. On peut choisir une et seulement une de ces positions possibles.

Remarquez également que nous avons souligné le mot ASC, pour indiquer qu'il s'agit d'une ***valeur par défaut***, c'est-à-dire une valeur prise par le système lorsque l'utilisateur ne choisit pas une option différente de l'ensemble des choix disponibles.

```
SELECT nom_colonne_1, nom_colonne_2, ..., nom_colonne_n
FROM nom_table
[ WHERE condition ]
[ ORDER BY nom_colonne_1 [ASC | DESC] [, nom_colonne_2 [ASC | DESC], ...] ];
```

figure III.2 Clauses obligatoires et facultatives indiquées explicitement dans l'instruction SQL

Sur la *figure III.2*, à chaque fois que la clause ORDER BY est mise en œuvre et que vous ne choisissez pas l'option DESC, le SGBDR prend l'option ASC par défaut.

Lorsque vous écrivez des instructions ou commandes SQL, il est intéressant de suivre certains principes pour améliorer la lisibilité des instructions et faciliter leur édition lorsque cela s'avère nécessaire. Vous devez en particulier avoir à l'esprit les points suivants :

- Les instructions SQL ne sont pas sensibles à la casse. Toutefois, les mots clés qui débutent par une clause sont généralement écrits en majuscule (il s'agit pas d'une contrainte) pour améliorer la lisibilité des instructions SQL ;
- Les instructions SQL peuvent être écrites sur une ou plusieurs lignes. Il est habituel d'écrire chaque clause sur une ligne distincte ;
- Un mot clé ne peut être réparti sur deux lignes et, à quelques exceptions près, ne peut être abrégé ;

Les instructions SQL se terminent par un point-virgule ; il doit suivre la dernière clause de l'instruction mais il n'est pas nécessaire qu'il soit sur la même ligne.

III- Conventions de noms des objets de base de données

Les objets de base de données, à savoir les noms des tables et de leurs attributs, doivent obéir à certaines règles ou conventions qui varient d'un SGBDR à un autre. Ne pas accepter les conventions de noms d'un SGBDR donné peut provoquer des erreurs. Par contre les utilisateurs sont généralement « ***protégés*** » s'ils respectent les indications suivantes :

- Les noms doivent comporter 1 à 30 (64 dans MySQL) ; toutefois, les noms des bases de données peuvent être limités à 8 caractères, comme c'est le cas pour des bases de données Oracle ;
- Les noms doivent commencer par une lettre (minuscule ou majuscule) ; les autres caractères peuvent comporter toute combinaison de majuscule, minuscule, chiffre ou traits de soulignement "_". Les espaces entre les noms sont à éviter.

Exemple II.1

Indiquer si ces noms sont généralement permis selon la convention des noms SQL. S'ils ne sont pas valides, expliquez pourquoi.

a- Noms de base de données : *Employe, CommandesAnneé, Données ventes*.
b- Noms de tables : *PIECES, Date_Naissance, Nom-vendeurs*. c- Noms d'attributs : *No_Id_Ordinateur, $_Valeur, ruevillecodepostalpays*.

Solution II.1 a- Noms de base de

données :

- *Employe* -> correct ;
- *CommandesAnnée* -> incorrect : Parce que dans de nombreuses bases de données de SQL, les noms de bases de données ne doivent pas dépassés 8 caractères ;

- *Données ventes* -> incorrect : Parce que les espaces ne sont pas autorisées.

b- Noms de tables :
- *PIECES* -> correct (ainsi que *pieces*) ; o *Date_Naissance* -> correct ;
- *Nom-vendeurs* -> incorrect : Parce que seuls les lettres, chiffres et le trait de soulignement sont permis.

c- Noms d'attributs :
- *No_Id_Ordinateur* -> correct ;
- *$_Valeur* -> incorrect : Parce que les caractères autres que les lettres, chiffres et le trait de soulignement ne sont pas permis.
- *ruevillecodepostalpays* -> correct mais très astucieux. Non seulement il est difficile de lire sans trait de soulignements entre les mots, mais cet attributs suppose que plusieurs éléments se trouvent dans cette colonne. Choisissez vos colonnes de sorte qu'un seul élément de données soit stocké dans chacune d'elles pour chacun des enregistrements. En fait, cet attribut devrait être décomposé en quatre colonnes : *rue, ville, codepostal et pays*.

Bibliographie et Webographie

Georges Gardarin, *Bases de données*, Eyrolles, Paris, 2003.

Philippe Rigaux, *Cours de bases de données*.

Thierry Lecroq, Bases de données.

Rim Chaabane, *Bases de données*, Université Paris 8.

Stéphane Crozat, Conception des Bases de données I – Introduction aux bases de données relationnelles.

Maude Manouvrier, Systèmes de Gestion de Bases de Données (SGBD) relationnels.

Antoine Spicher, Base de données Modèle Entité/Association.

Philippe Gambette, Base de données Modèle Entité/Association.

www.developpez.net

www.openclassrooms.com